Information Retrieval

WS 2022 / 2023

Lecture 3, Tuesday November 8th, 2022
(Efficient List Intersection, Lagrange Multipliers)

Prof. Dr. Hannah Bast
Chair of Algorithms and Data Structures
Department of Computer Science
University of Freiburg

UNI FREIBURG

Overview of this lecture

- Organizational
  - Your experiences with ES2 — Ranking and Evaluation
- Contents
  - Recap and motivation — Intersect vs. Merge, Timing
  - Engineering the Zipper algor. — All kinds of tricks, PyPy, C++
  - Other algorithms — Galloping-Search Intersect
  - Lagrange multipliers — The **math** needed for ES3

ES3: Analyze the running time of intersection using galloping search, practicing the important "Lagrange multipliers" method

Experiences with ES2 1/3

UNI FREIBURG

- Summary / excerpts
  - Interesting exercise, learned a lot, quite time-consuming for most (but two weeks time), many small problems, some difficulties understanding the doc tests

    "I had a really great time with this assignment"

    "I've lost track of the time, too many little problems"

    "Mixed experience: first fun … then trouble passing tests"

    "Very interesting and challenging exercises, but would have liked more hints on how to use the functions from the test"

    "Knowing the datatypes of the test cases would be helpful"

    "I felt very productive and it was fun, but afterwards I am always surprised how much time it took"

    "I don't watch movies, I like books"

Experiences with ES2 2/3

- Results
  - Changes improve some queries and make others worse
  - Small differences in the implementation can make a significant difference in the results
  - Smaller values for b gave the best results for most
  - Baseline: MP@3 = 73%, MP@R = 54%, MAP = 55%
  - Best results: MP@3 = 85%, MP@R = 60%, MAP = 61%

Bottom line: tuning a ranking algorithm is super important (for result quality) but also super hard

In particular, it is very hard to understand / predict the effect of changes in the parameters / implementation

Experiences with ES2 3/3

- Your favorite movies
  - Some of your favorites among the Top-100:

    Inception (#3), Forrest Gump (#5), The Matrix (#7), Interstellar (#11), American History X (#42), …
  - Some of your favorites ranked beyond 100:

The Lives of Others (#426)	German movie, but good
Predestination (#654)	deserves much better
The Rocky Horror Picture Show (#1430)	why so low
Spirit: Stallion of the Cimarron (#2723)	great music
La Dolce Vita (#2831)	by Federico Fellini
Fate/stay night: Heaven's Feel III (#24276)	アニメはほんとうにすばらしいです

Recap and Motivation 1/2

- List intersection
  - In Lecture 1, we have **intersected** the inverted lists
  - In Lecture 2, we have **merged** the inverted lists
  - Both modes are relevant for and used in real search engines
  - Intersection and merging of (sorted) lists of integers is also a basic operation in database engines

    An "inner join" corresponds to an "intersect" and an "outer join" corresponds to a "merge" … see Lecture 12
  - Today we will focus on **efficient** list intersection

Recap and Motivation 2/2

- A note on time measurement
  - Trickier than it may seem at first, because there can be significant variation between runs, for example due to:

    Other jobs running on your machine

    The garbage collector running unpredictably (in Java)

    Which part of the data is in the (L1 / TLB / disk) cache
  - For today's lecture, we **repeat** each measurement **three times** to at least get a feeling for the variation

    Note: the repetition itself can distort the truth because an earlier run may put data in the cache, which then benefits later runs

Engineering the Zipper algorithm 1/6

- Our plan
  - Recall: By "Zipper" we mean our simple linear-time algorithm from Lecture 1 for intersecting two sorted lists
  - We will look at the following variations today:

    Small variations in the algorithm (still linear time)

    Variations in how the lists are stored (list, array, numpy)

    Small variations in the implementation

    Different "compilers" (python3 vs. pypy3)

    Different programming languages (Python vs. C++)

    What do you think will be the difference between our slowest and our fastest code?

Engineering the Zipper algorithm 2/6

- Small variations in the algorithm
  - Reduce the code that is executed in loops

    Making the loop body of Zipper x% faster will make the whole algorithm x% faster
  - Improve the branch predictability

    Processors tentatively execute the next instruction before the result of the current instruction is known, which requires "guessing" the outcome of conditionals
  - Add sentinel (value ∞) to the end of both lists

    That way we don't have to test after each comparison whether we have already reached the end of one list

    Altogether, these can make our Python code **5** times faster

Engineering the Zipper algorithm 3/6

- Variations in how the lists are stored (Python)
  - **list**: Python's built-in data structure, different elements can have different types, optimization when all elements have the same type
  - **array.array**: array of elements that all have the same type (which has to be specified at construction time)
  - **numpy.array**: optimized for linear-algebra computations, but slow when used as a general-purpose array; also, numpy arrays cannot change their size efficiently

For our simple algorithm, Python's built-in lists are actually better than array.array or numpy.array

Engineering the Zipper algorithm 4/6

- Python vs. PyPy
  - PyPy wants to be a drop-in replacement for Python that is faster due to Just-In-Time (JIT) compilation

    python3 intersect_timing.py 1000 1000000

    pypy3 intersect_timing.py 1000 1000000
  - Works for a large subset of Python, but not fully compatible

    For example, pattern matching in Python 3.10
  - PyPy is written in RPython (restricted Python), and the RPython interpreter is written in Python

    Hence the logo: a snake swallowing itself

    For our code: improvement by a factor of almost 100

Engineering the Zipper algorithm 5/6

- C++
  - In principle, C++ allows programming with type-safety and nice abstractions, without loss of performance compared to C

    In practice, some of the C++ syntax and concepts are not ideal because C++ has been around for so long and has to be backward-compatible with earlier (suboptimal) versions

    However, for our Zipper algorithm, the C++ code looks very similar to the Python code

    For our code: around 200 times faster than Python, and still around 5 times faster than PyPy

    Altogether: our fastest C++ program is around **1000 times faster** than our first Python program

Engineering the Zipper algorithm 6/6

- Bottom Line
  - There is **huge** potential for performance gains even when using the same basic algorithm
  - But you need an in-depth understanding of your programming languages and how the code you write maps to machine code

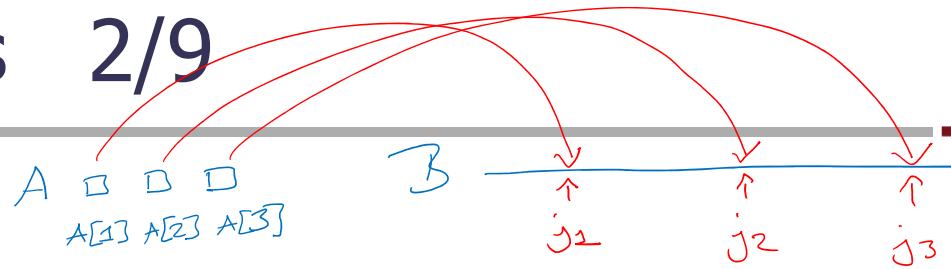
Other algorithms 1/9

- Preliminaries
  - We have two lists, which we want to intersect
  - Let **A** be the smaller list, with **k** elements $k \leq n$
  - Let **B** be the longer list, with **n** elements

    List intersection is commutative, so we can always assume that the first list is A, and the second is B

    If k = n, it does not matter which list is A and which is B
  - We denote the elements of **A** by A[1], …, A[k]
  - We denote the elements of **B** by B[1], …, B[n]
  - Recall that both lists are **sorted** … this is crucial for the basic algorithm and all the algorithms in the following

Other algorithms 2/9



- General idea
  - All of the following algorithms try to **locate** the elements of the smaller list A in B
  - That is, they compute $j_1, \ldots, j_k$ such that for each j_i

    $B[j_i] \leq A[i]$ if $j_i > 0$ (otherwise $A[i] < B[1]$)

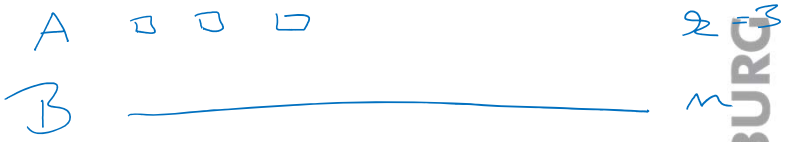
    $A[i] \leq B[j_i+1]$ if $j_i < n$ (otherwise $A[i] > B[n]$)

    The idea here is that A[i] is placed <u>after</u> $B[j_i]$ in the merged list (or at the beginning if $j_i = 0$)
  - Given such $j_1, \ldots, j_k$ we can easily compute the intersection of A and B in O(k) time

    Just check whether A[i] is equal to $B[j_i]$ or $B[j_i+1]$

    (depending on how exactly the algorithm orders equal elements in the merged result)

Other algorithms 3/9



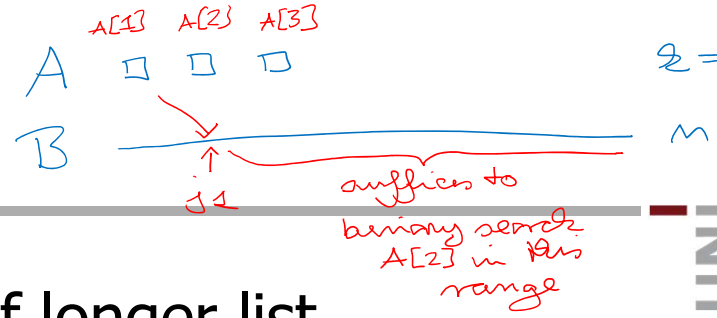
- Binary search in the longer list
  - For each $i = 1, \ldots, k$ search $A[i]$ in $B[1..n]$

    Use binary search for each search (note that B is sorted)
  - This has time complexity $\Theta(k \cdot \log n)$

    This is good for small k

    But for $k = \Theta(n)$ this is $\Theta(n \cdot \log n)$, and hence slower than the linear-time Zipper algorithm

Other algorithms 4/9



- Binary search in **remainder** of longer list
  - For each $i = 1, \ldots, k$ search $A[i]$ in $B[\, j_{i-1} \,..\, n\,]$

    Again, using binary search (in the part of B after A[i-1])
  - Time complexity in the "**best**" **case**: $\Theta(\log n)$

    For example: first element of A is at the end of B

  - Time complexity in the **worst case**: $\Theta(k \cdot \log n)$

    For example: all elements of A are at the beginning of B

  - Time complexity in the **"typical" case**: $\Theta(k \cdot \log n)$

    For example: elements of A are evenly distributed over B

$$\log n + \log\left(n - \frac{n}{k}\right) + \log\left(n - 2\frac{n}{k}\right) + \cdots$$

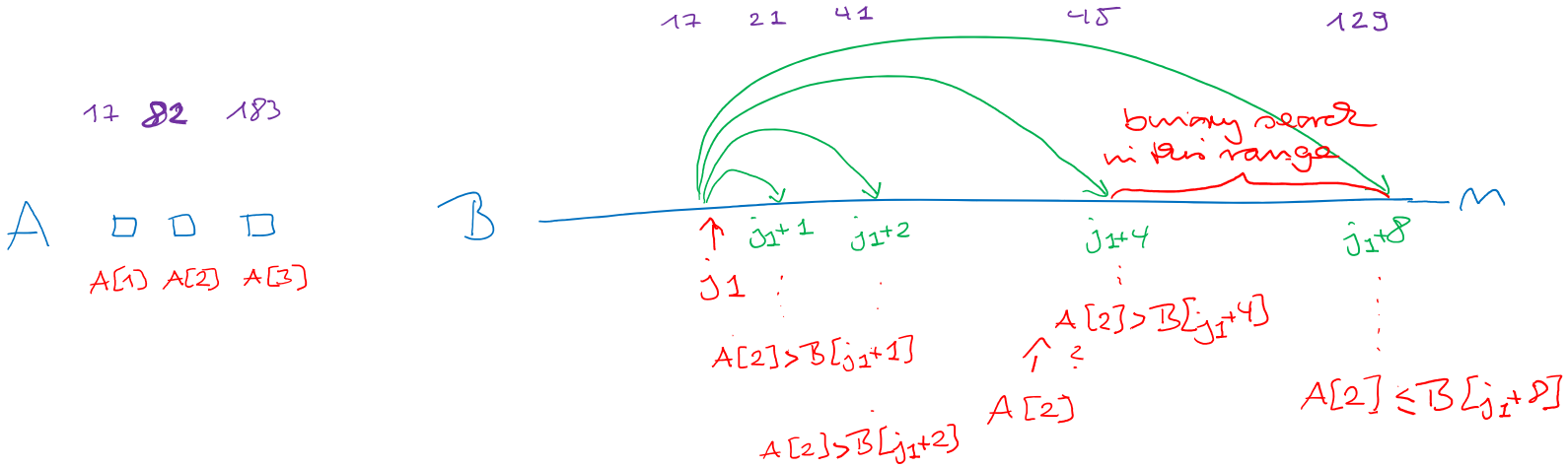
the first $k/2$ of these are still searching in more than $\frac{n}{2}$ of B

Other algorithms 5/9

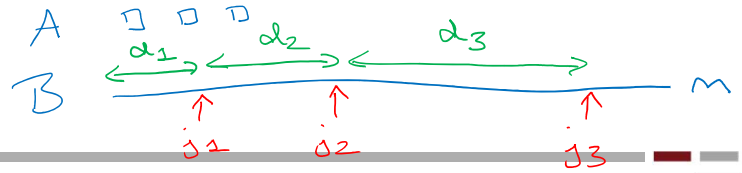
- **Galloping** search in remainder of longer list
  - For each $i = 1, \ldots, k$ find the position j_i of **A[i]** in **B** using an exponential search followed by a binary search

    Exponential search: compare **A[i]** to the elements of **B** at pos. $j' = j_{i-1} + 1, j_{i-1} + 2, j_{i-1} + 4, j_{i-1} + 8, \ldots$ until $B[j'] \geq A[i]$

    Then binary search **A[i]** in **B[j''+1 .. j']**, where j'' is the position before j' in the exponential search (where $B[j''] < A[i]$)



Other algorithms 6/9



$\log_2(2d_3) = \log_2 d_3 + 1$
$= \Theta(\log d_3)$

- Galloping search, continued
  - **Lemma:** Using the exponential-binary search described on the previous slide, j_i can be computed in time $\Theta(\log d_i)$, where $d_i = j_i - j_{i-1}$ for $i > 1$ and $d_1 = j_1$

    ES3: Work out the details and prove that this can indeed be done in time $\Theta(\log d_i)$

Other algorithms 7/9

$\sum_{i=1}^{k} \log d_i \leq \sum_{i=1}^{k} \log \left(\frac{n}{k}\right)$

CONSTRAINT: $\sum_{i=1}^{n} d_i \leq n$

- Galloping-search intersect, total time complexity
  - The total time complexity is $O(k + \Sigma_i \log d_i)$

    ES3: Understand why we need the k + here
  - We want a formula that only depends on k and n
  - Lemma: $\Sigma_i \log d_i$ is maximized when all $d_i = n / k$

    1+ ... because log 0 is not defined

    ES3: Prove this using Lagrange multipliers → slides 23 – 28
  - Galloping search therefore takes time $O(k \cdot \log (1 + n/k))$

    ES3: Prove that this is asymptotically never worse than Zipper

Note: for $k = \Theta(1)$, the bound is $O(\log n)$

Other algorithms 8/9

- Comparison-based lower bound
  - Recall the lower-bound for comparison-based sorting

    There are n! possible outputs, we have to differentiate between all of them, and only two choices per step

    Hence #steps required $\geq \log_2 (n!) = \mathbf{\Omega(n \cdot \log n)}$
  - We can use a similar argument for intersection / union:

    There are n + k over k ways how the k elements from A can be placed within the n elements from B, …

    Hence #steps required $\geq \log_2 (n/k)^k = \mathbf{k \cdot \log_2 (n/k)}$

  Galloping search is hence asymptotically optimal

Other algorithms 9/9

- Skip Pointers
  - **Idea:** potentially skip large parts of longer list B
  - Skip pointer = special element in list B with a value x and the index j of the first element in B with $B[j] \geq x$

    When intersecting, follow pointer if current $A[i] \geq x$
  - Advantage: **very simple** to implement

    In particular, simpler than galloping search and sometimes more effective in practice, even if not "optimal"

Lagrange multipliers 1/6

- Problem formulation
  - Find the maximal value of $f(x_1, \ldots, x_n)$ under the constraint $g(x_1, \ldots, x_n) = 0$ for all $x_1, \ldots, x_n \in \mathbf{R}$
  - Assumption: Both f and g have a continuous first derivative and not all $\partial g / \partial x_i = 0$ for $x_1, \ldots, x_n$ with $g(x_1, \ldots, x_n) = 0$
- Algorithm
  - Define the following "Lagrangian" function based on f and g:

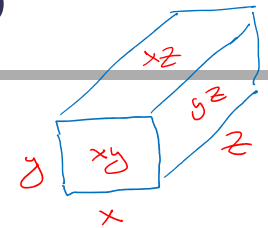
    $L(x_1, \ldots, x_n, \lambda) := f(x_1, \ldots, x_n) - \lambda \cdot g(x_1, \ldots, x_n)$
  - Set all n + 1 partial derivatives to zero and find the solutions

    That is, find all solutions of: $\partial L / \partial x_i = 0$ and $\partial L / \partial \lambda = 0$
  - Evaluate f at all these solutions and pick the maximal one

    If there is only one solution, check that it's a maximum by computing the value of f at some other point

Lagrange multipliers 2/6

$$2xy + 2xz + 2yz = 6$$
$$xy + xz + yz = 3$$
$$\underbrace{xy + xz + yz - 3}_{=:g(x,y,z)} = 0$$



- Example
  - Assume we have a box (cuboid) with side lengths x, y, z

    Note: this has volume $x \cdot y \cdot z$ and surface $2xy + 2xz + 2yz$
  - If we want to maximize the volume under the constraint that the surface is **exactly** 6m², we can formulate this as:

    Find the maximum value of $f(x, y, z) = x \cdot y \cdot z$ under the constraint that $g(x, y, z) = xy + xz + yz - 3 = 0$
  - Check the assumptions:

    $(\partial f/\partial x, \partial f/\partial y, \partial f/\partial z) = (yz, xz, xy)$ … continuous

    $(\partial g/\partial x, \partial g/\partial y, \partial g/\partial z) = (y+z, x+z, x+y)$ … continuous and $\neq 0$ for $g(x, y, z) = 0$

Lagrange multipliers 3/6

- Example, continued
  - Let's write down the Lagrangian for this example:

    $L(x, y, z, \lambda) = x \cdot y \cdot z - \lambda \cdot (xy + xz + yz - 3)$

  - Now let's find all solutions, where all four partial derivatives $\partial L/\partial x$, $\partial L/\partial y$, $\partial L/\partial z$, and $\partial L/\partial \lambda$ are zero

(I) $\frac{\partial L}{\partial x} = yz - \lambda y - \lambda z = 0$

(II) $\frac{\partial L}{\partial y} = xz - \lambda x - \lambda z = 0$

(III) $\frac{\partial L}{\partial z} = xy - \lambda x - \lambda y = 0$

(IV) $\frac{\partial L}{\partial \lambda} = -(xy + xz + yz - 3) = 0$

(I) $\wedge$ (II): $(x-y)z = (x-y)\cdot\lambda$

$\Leftrightarrow x = y \vee z = \lambda$

similarly:

$x = z \vee y = \lambda$

$y = z \vee x = \lambda$

only solution:

$x = y = z$, $\lambda = \frac{1}{2}$

Lagrange multipliers 4/6

$f(x) = x \cdot y \cdot z$

$g(x) = xy + yz + xz - 3 = 0$

- Example, continued
  - Let's compute the value of f at this solution and since it's the only one, let's evaluate f at one other point to check that it's a maximum

$x = y = z \overset{g(x,y,z)=0}{\Longrightarrow} x = y = z = \underline{1}$

$f(x,y,z) = \underline{1}$

Lagrange multipliers 5/6

- When the side constraint is an **in**equality
  - The Lagrangian multiplier method does **not** directly apply when the constraint is an **in**equality, for example:

    maximize $V = x \cdot y \cdot z$

    under the constraint that $2xy + 2xz + 2yz \leq 6$
  - However, we can solve

    maximize $V = x \cdot y \cdot z$

    under the constraint that $2xy + 2xz + 2yz = \mathbf{S}$

    and then maximize the solution under the constraint $S \leq 6$ (where the maximum is trivially achieved at $S = 6$)

    The same "trick" also works for ES3

Lagrange multipliers 6/6

- When there are additional constraints
  - In our example, we have computed the maximum for any $x, y, z \in \mathbf{R}$ … including negative values

    That was fine because we found a maximum even when taking the negative values into account
  - For ES3, the variables must be non-negative numbers, since otherwise the function f, which involves the logarithm, is not defined

    You can simply ignore this complication for the exercise sheet and assume that the method described on the previous slides still finds the maximum

References

- Textbook

  Section 2.3: Faster intersection with skip pointers

- Literature

  A simple algorithm for merging two linearly ordered sets

  F.K. Hwang and S. Lin SICOMP 1(1):31–39, 1980

  A fast set intersection algorithm for sorted sequences

  R. Baeza-Yates CPM, LNCS 3109, 31–39, 2004

- Wikipedia

  http://en.wikipedia.org/wiki/Lagrange_multiplier